DeepSeek DeepInfer: a source-level systems review

Released runtime, execution boundaries, and engineering implications

- DeepInfer snapshot `9bd9cf243fc53ef9afbf8f04d391d989117d6ab9`
- Reviewed 23 October 2026 for AI and data infrastructure engineering leaders
- Companion repositories provide context. Most slides trace inference implementation.

> 重点读实际执行路径。代码能证明的实现、官方部署文档、工程推断分别说明。性能数据没有在本机复测。

DeepInfer release scope

The release boundary

Offline inference includes model execution and local cache machinery

Released code	Outside the complete public service
Scheduler, model, sampler, local GPU KV/state allocation	External KV cache system and storage lifecycle
Attention DP/CP, MoE EP, expert placement and shuffle	VE/prefill/decode service orchestration
Async worker steps, speculative decoding, output constraints	Routing and load balancing across service replicas

> README 说缺 KV cache system，不代表没有 GPU KV 管理。必须区分本地执行内存与跨机器、可持久化的缓存服务。

Explicit release boundary Integration hooks

Repository inventory by system layer

The recent runtime release builds on earlier infrastructure libraries

Layer	Official repositories	Review depth
Runtime and API protocol	DeepInfer, deepseek-recipe	Code paths reviewed
Kernels and toolchain	FlashMLA, DeepGEMM, DeepEP, DeepSelect, TileKernels, DeepJIT	DeepInfer call sites and pinned gitlinks
Ascend and speculative ecosystem	DeepGEMM-Ascend, DeepEP-Ascend, DeepSpec	Inventory context only
Storage, data processing, training	3FS, smallpond, DualPipe, EPLB	Earlier 2025 release family
Agent tools	deepseek-harness	Search/filesystem seams inspected

> 2026 年新发布的推理组件和 2025 年开源周项目不是同一批。这里列相关系统仓库，不把所有模型权重仓库混入。

Official infra index Vendor dependencies Official organization Inventory snapshot

“Search” and “filesystem” have distinct meanings

No standalone open search backend has been established in this review

Code location	What the public source implements
harness web-deepseek-api/src/search.ts	An HTTP adapter calling /v1/tools/search
harness tool-fs-search/src/search-core.ts	Local glob/grep discovery using packaged ripgrep
harness fs-local / fs-sandbox / fs-observation-policy	Agent file access, mutation containment, freshness policy
deepseek-ai/3FS	Distributed storage, separate from agent file tools
DeepSelect	GPU TopK selection for attention and sampling

> 用户说的 search 仍可能指其他项目。已验证的 web search 是 API 客户端，不是爬虫或索引后端。文件系统工具与 3FS 也不能混为一谈。

Search provider code ripgrep execution Filesystem contract 3FS

The public entry point

LLM.generate adds every request before driving the engine loop

```
LLM.generate(...)
  for i in range(num_requests):
      self._add_request(...)
  return self._run_engine(use_tqdm)
```

- `_add_request` copies sampling parameters, tokenizes if needed, converts IDs to int32.
- The offline example configures EP/DP/CP, memory fraction, token budget and sequence cap.

> 这不是 HTTP server。批量输入由 Python API 注入 engine，适合离线推理与集成验证。

LLM.generate / _add_request Offline configuration

Request and token execution flow

One request moves through admission, metadata, forward, sampling, and reclamation

Stage	Concrete implementation
Input conversion and Sequence creation	LLM._add_request / LLMEngine.add_request
Batch selection and metadata	Scheduler.schedule in scheduler.hpp
Device execution and previous-step retirement	Worker.execute_model_step
Main model / sampler	ModelRunner.execute_main_model / execute_sample
State update and next token output	LLMEngine.post_process / Scheduler.process_outputs
Final drain	LLM._run_engine / flush_inflight_reqs

> 按调用链读代码比按目录猜架构更可靠。step 返回的结果在异步路径可能属于上一个 batch。

Engine step Worker loop

Batch type requires group agreement

NORMAL engines vote on prefill versus decode across DP ranks

```
// Scheduler._schedule, condition shown exactly
bool want = (get_gpu_cache_usage() <= GPU_CACHE_PREFILL_WATERMARK)
            && _has_prefill_waiting();
// ANY: one rank wants prefill, ALL: every rank wants prefill
// Second vote: a rank actually scheduled work.
```

- The watermark is `0.9`. Configured intervals create ALL and ANY attempt points.
- Idle ranks execute a placeholder prefill step when another rank has work.
- A second vote falls back to decode when admission fails everywhere.

> 这保证 collective 的执行步型一致。ANY 尝试给孤立 rank 兜底，第二次投票避免 waiting 有请求却无法分配时反复空跑。

Scheduler._schedule Watermark

Prefill admission and chunking

The queue head must satisfy memory, sequence, and token constraints

```
new_workload = std::min(seq_workload,
    (max_num_batched_tokens - num_batched_tokens) / cp_align * cp_align);
```

- `_schedule_prefill` scans waiting requests from the front and stops at an unschedulable head.
- `AllocStatus::LATER` waits for blocks. `NEVER` rejects a request that exceeds total capacity.
- The remaining batch-token budget bounds each chunk, aligned to CP requirements.
- The scheduler also limits prompt count and aggregate key length. Forked prefixes require block/state cloning.

> 公开版本这段 admission 主要受 token、slot、KV 限制。部署 Q&A 描述的跨实例 compute-quota 路由不能当作这里已经完整实现。

Scheduler._schedule_prefill

Decode admission and preemption

Memory pressure can terminate requests or send them back for recomputation

- The scheduler admits prompted requests and prepares forked state and tail blocks.
- It sorts running sequences by output length, visits the short-output end, and preempts from the opposite end when blocks run out.
- `_append_slot` grows each admitted sequence, then `MTPVarlen.schedule` chooses verification lengths.
- `_preempt` stops structured-output requests and stop-on-preemption configurations. Other requests reset and re-enter waiting.

> 这里的排序键是已生成长度，不是预测完成时间。COMPLETION 引擎没有 prefill 通道，构造时强制 stop_preempted_request=true。

Scheduler._schedule_decode Scheduler._preempt Completion invariant

Asynchronous scheduling overlaps retirement

The worker launches current forward before retiring previous sampling results

```
# Worker.execute_model_step, order abridged
main_model_output = self.model_runner.execute_main_model(...)
request_outputs = self._post_process_step(self.prev_step_state)
sampler_output = self.model_runner.execute_sample(...)
# ZOS decode defers current post-process to the next step
self.prev_step_state = curr_step_state
```

- Prompt and non-ZOS steps retire immediately. ZOS decode keeps one previous step in flight.
- `execute_main_model` and `execute_sample` wrap their device launch sections in `forbid_cuda_sync()`.

> “zero overhead”是优化目标，不是没有 CPU 工作。关键是 CPU bookkeeping 与下一次 GPU forward 重叠，而不是神奇地消失。

Worker.execute_model_step Launch synchronization boundary

Scheduling ahead requires device corrections

CPU sequence lengths can lag the most recent sampling decision

```
compute_seqlens_q_by_confidence_rates(... confidence_rates_gpu ...)
compute_seqlens_k_zos(... num_accepted_tokens_gpu ...)
seqlens_q_cpu[:n_seq].copy_(seqlens_q[:n_seq], non_blocking=True)
```

- `prepare_zos_inputs` maps overlapping sequences between the current and previous batch.
- GPU kernels use prior acceptance counts to correct key lengths and confidence rates to adjust query lengths.
- Varlen query lengths copy back asynchronously. The worker waits for that copy before updating CPU sampling metadata.
- The final drain explicitly synchronizes and processes remaining in-flight work.

> 异步优化把依赖转成显式 remap 和事件。集成时不能绕开 flush，也不能把 CPU cache.num_tokens 当作最新 GPU 状态。

Worker._correct_seqlens_qk Retirement and drain C++ ZOS input preparation

Object lifetime is part of correctness

Pooled outputs and borrowed vectors need explicit ownership rules

- `LLMEngine.step` documents that RequestOutput objects expire on the next step.
- The offline loop clones each output before retaining it.
- Scheduler outputs borrow clone/zero vector storage that the next `schedule()` clears.
- Deferred release collects metadata, outputs, and block/state unpins across worker execution.

> 对外做 streaming wrapper 时尤其容易踩这个坑：必须复制或明确转移生命周期，不能把池化指针异步留给网络线程。

Output lifetime contract Offline output cloning Borrowed vector contract DeferredRelease

Local block allocation and sharing

BlockManager tracks free blocks and reference counts under a lock

```
int allocate() {
    int block_id = _pop_front();
    _blocks[block_id].ref_count += 1;
    return block_id;
}
// pin increments the count, free returns a block at count == 0
```

- Block ID 0 is reserved as an empty sentinel.
- Sequence block tables reference physical storage. Fork paths pin shared data and clone writable tail blocks.
- A separate state manager reserves recurrent/compression state slots alongside KV blocks.

> 公开代码能读到显存内 block 分配和共享语义，但这不等于前缀 hash 索引、跨实例恢复、存储 GC 都已开源。

Blocks / BlockManager Fork cloning State allocation check

KV memory includes multiple representations

Cache sizing follows attention layout, indexer precision, and rollback history

Representation	Code-level sizing or mapping
Main interleaved KV	V41_FORMAT / V41_FP4_FORMAT include data and scale bytes
Indexer K cache	FP4 channels plus one UE8M0 scale per 32 channels
Sliding-window state	window_size + MAX_BACKTRACKING_TOKENS
Compression state ring	compress_ratio + max(0, backtracking - 1)
Shared layers	layer_kv_map and index maps reuse earlier layer storage

> 不能直接套传统 MHA 的 layers×heads×head_dim 公式。这里有压缩、量化、状态环和跨层共享，容量规划必须按实际模型配置计算。

KV bytes and ring sizing Layer sharing maps

Bounded replay compacts consumer work

YoCo keeps a bounded suffix where the model structure permits it

- The model configuration divides producer and consumer layers at `n_prefill_layer`.
- `yoco_boundary_compact` gathers kept query rows. With CP, it gathers contributions and redistributes compact rows.
- Consumer metadata switches query-aligned fields and the SWA visibility domain.
- README defaults use compact mode and 140 retained tokens. The suffix must cover SWA plus maximum backtracking.

> 这不是任意模型都能截断 prefix。依赖 YoCo 层结构和恢复状态，140 是该版本 V4.1 配置，不是通用窗口。

YoCo boundary operations Consumer metadata Release configuration

Attention and experts use different groups

DP, CP, and EP compose around the model execution path

Group	Implementation role
Attention DP	Separate request streams, synchronized batch-type decisions
Balanced CP	Distributes context work, uses gather and broadcast helpers
MoE EP	Contiguous expert groups exchange routed token activations
CPU groups	Gloo groups coordinate host-side metadata and decisions
Shared-memory queue	CP metadata broadcast when CP size exceeds one

> 源码支持 CP，部署 Q&A 同时说当前生产配置未使用 TP/CP。支持能力与具体线上配置需要分开看。

ParallelProcessGroup.create_process_groups CP all-gather buffer Deployment choices, documentation

MoE dispatch, expert compute, and combine

The forward path explicitly coordinates communication and shared experts

```
topk_idx, topk_weights = self.gate_forward(...)
event_overlap = self.dispatch(...)
moe_in_hidden, ..., handle = event_overlap.current_stream_wait()
moe_out_hidden = self.routed_experts_forward(...)
combine_pre_evt = self.ep_buffer.capture()
shared_out = self.shared_forward(hidden_states)
event_overlap = self.combine(..., shared_out, handle, combine_pre_evt)
```

- DeepEP owns token exchange and handles. Routed expert weights use rank-local physical expert rows.
- The standard path limits communication SM use while computing shared experts. A separate mega-MoE branch exists.

> 通信开销与计算并行设计绑定。单看 GEMM 吞吐不能推导整个 MoE step 的收益。

MoE.forward dispatch / combine Expert parameter layout

EPLB balances physical expert replicas

The policy uses historical per-step load rather than only total token counts

- `rebalance_experts_global` allocates redundant replicas to hot logical experts.
- `_batch_greedy_indices` places physical experts under per-rank capacity constraints.
- The placement objective sums the per-step maximum rank load over the history window.
- Shuffle code offers batched P2P and an intranode symmetric-memory/IPC path. This differs from request routing across replicas.

> EPLB 改 expert 布局，DP load balancer 改请求去向。两者优化不同失衡来源，不能因为有 EPLB 就宣称公开了完整服务路由。

Replica selection and greedy placement Weight shuffle paths Intranode shuffle

Attention combines SWA and compressed KV

Prefill and decode use different data paths into sparse attention kernels

Phase	Observed implementation
Prefill	Builds full/SWA and compressed indices, calls prefill_attn_varlen_sparse
Decode	Reads paged swa_cache and ca_cache, calls decode_mla_mqa_sparse
Projection	Fused norm/RoPE/cast plus DeepGEMM output projection
Metadata	Kernel scheduling metadata and index lengths vary by attention configuration

> 这里描述的是 V4 模块实现，不把 V3 的 MLA 简图直接套过来。SWA、压缩索引和共享层会改变数据可见范围。

Prefill and decode attention Fused projection operations

Indexer logits feed TopK attention selection

DeepSelect selects token indices inside the model

```
# MQA._topk_prefill, abridged
p_sum = self._compute_prefill_p_sum(...)
topk(p_sum, self.topk_tokens, sorted=False,
     end=topk_lens[chunk], sorted_index=True,
     output_idx=topk_index[chunk],
     idx_oob_fill_value=-1, return_value=False)
```

- DeepGEMM computes dense or sparse FP8/FP4 MQA logits.
- Prefill chunks the query computation to bound temporary score storage. Decode uses paged metadata.
- Candidate-block paths restore token indices after TopK selection.

> 模型内部的检索/TopK 与 web search 是不同概念。这里返回注意力 token 索引，不是文档搜索结果。

MQA._topk_prefill Paged decode logits

Kernel dependencies are pinned separately

DeepInfer records exact vendor gitlinks rather than bundling all implementation

Dependency	Pinned revision	Role at call sites
FlashMLA	2e5429fc5653	Attention kernels and fused transforms
DeepGEMM	1fc9c66975ee	Low-precision matrix and indexer logits
DeepEP	f6aa38fe06cd	Expert token dispatch/combine
DeepSelect	bfa4507d935f	TopK selection
Ascend variants	DeepEP 3b25377d04b2 DeepGEMM 3a35326271b0	Platform-specific dependencies

> 本次深读重点是 runtime 与库的接口，不声称逐行审计所有 vendor kernel。复现时必须递归获取这些锁定版本。

Pinned vendor tree Submodule URLs Kernel use

Graph buckets influence execution cost

The graph runner selects the smallest captured bucket that covers total_q

```
index = bisect_left(self.batch_sizes, total_q)
if index == len(self.batch_sizes):
    raise ValueError(...)
batch_size = self.batch_sizes[index]
return batch_size, self.graphs[batch_size]
```

- Capture stores reusable graph outputs and input buffers, sharing a memory pool.
- The replay path uses a graph handle for timing when supported.
- Bucket padding affects both execution cost and the speculative scheduler’s best batch choice.

> 有效 token 数与 graph bucket 大小是两个量。调参要同时看 padding、内存与测得的耗时曲线，不能只增加 batch 上限。

Capture / select / replay Graph-aware throughput estimate

Confidence scheduling chooses verification work

MTPVarlen maximizes expected accepted tokens per estimated step time

```
time_ms = baseline_ms + slope_ms * graph_size + sampler_ms;
cur_throughput = cur_expected / time_ms;
if (cur_throughput > best_throughput) {
    best_throughput = cur_throughput;
    best_bsz = cur_bsz;
}
```

- One golden token per sequence forms the baseline. Ranked confidence estimates add draft verification slots.
- N−2 confidence chooses total work. N−1 information allocates per-sequence lengths, with GPU correction under ZOS.
- Measured forward/sampler times update runtime estimates. No decode curve falls back to all available drafts.

> DSpark 的关键是 draft 和 verification 的联合系统设计。这里不是固定每个请求验证 5 个 token，nextn 只是最大草稿长度。

MTPVarlen.schedule Runtime feedback

A device kernel redistributes the token budget

The CUDA TileLang implementation ranks cumulative draft confidence

```
# Example, two sequences and two draft positions
conditional rates:  A [0.9, 0.8]   B [0.6, 0.9]
cumulative rates:   A [0.9, 0.72]  B [0.6, 0.54]
total_q = 5, golden tokens = 2, extra budget = 3
selected drafts: A1, A2, B1
seqlens_q: A = 3, B = 2
```

- Phase 1 gathers prior rows through `remap` and computes cumulative products. Fresh rows use zero rates.
- Phase 2 counts better candidates in parallel. Higher confidence wins, with lower flat index breaking ties.
- Each sequence gets one golden token plus its selected drafts. This is an explicit rank-counting kernel, not a full sort.

> 示例按代码手算，未运行 GPU。pairwise rank counting 的工作量随 batch 和 nextn 增长，适合结合实际小 nextn 和测量结果评估，不能凭算法名假设无限扩展。

CUDA confidence kernel Wrapper and reference

Output constraints interact with recovery

Determinism and structured decoding require explicit configurations

Feature	Released behavior and constraint
Deterministic sampling	README requires batch invariance, MTP varlen off, and fixed seq_seed on NVIDIA
Structured output	JSON schema, regex, or EBNF compile through XGrammar
Token enforcement	Matcher builds next-token vocabulary masks and supports rollback
Preemption	Structured-output state currently blocks recomputation and causes request termination

> “支持 JSON schema”不代表任何资源压力下都可透明重试。服务层要把 FINISHED_PREEMPTED 等结束原因准确传给上层。

Determinism settings Compiler.dispatch_and_compile Matcher mask and rollback Preemption constraint

Recipe supplies a protocol boundary

deepseek-recipe converts conversations and output streams around an inference backend

- Rust libraries and Python bindings handle API conversion, prompt rendering, token encoding, and response parsing.
- `PromptTokenizer.encode_ids` receives explicit special-token ranges rather than treating all matching text as control tokens.
- The caller supplies inference, tool execution, and HTTP transport. Constraint declarations still require backend enforcement.

> 要做兼容 API，tokenizer.encode(prompt) 之外还有协议语义。Recipe 是可复用边界，但不会自动把 DeepInfer 变成完整在线服务。

Recipe release scope Special-token-aware tokenizer

Benchmark scope and reproducibility

The published Pareto curves use different prefill and decode deployments

Reported setup	What an engineering comparison must retain
Prefill: 4 SM100 GPUs	Input length, cached prefix, batch size, YoCo clipping
Decode: 64 SM100 GPUs	Context length, request concurrency, nextn and acceptance assumptions
Each within one scale-up domain	Topology and communication differ from cross-domain deployment
examples/sweep/run_sweep.sh	Sweeps batch sizes with profiling disabled and emits JSONL

> 本 deck 不转抄图中曲线点，也不把离线结果当作线上 SLA。官方部署文档的集群规模、收入和约 80% 离线效率是陈述，不是本次验证结果。

Benchmark hardware Sweep inputs / output Deployment claims, documentation only

Deployment prerequisites and missing services

The released target is Linux with recent datacenter accelerators

- Supported model: DeepSeek-V4.1-Flash. Hardware: NVIDIA SM100 or Huawei Ascend 950.
- README requires GCC ≥13, Python 3.12, CUDA 13 or CANN 9.2. A Mac cannot validate device execution.
- A full service still needs request routing, cache lookup/storage lifecycle, inter-stage transfer, cancellation, and API transport.
- Published Q&A describes external KV storage on 3FS and DP balancing. Those descriptions do not establish complete released source.

> 平台适配不等于任意显卡兼容。上线评估应先建立硬件、模型、锁定依赖和失败恢复矩阵，再考虑吞吐优化。

Hardware / prerequisites Storage and routing, documentation Unimplemented service hooks

Engineering takeaways

The strongest reusable ideas lie at state and execution boundaries

Observed design	Concrete evaluation work
GPU correction of speculative metadata	Test abort, batch membership change, final drain, and rollback together
Separate logical and physical experts	Measure hot-expert skew and redistribution cost at intended EP size
Compressed/shared cache layouts	Budget HBM from actual formats and backtracking requirements
Graph-aware verification budget	Profile graph buckets and sampler cost under realistic concurrency
Public service boundary	Estimate external cache and routing integration before committing to adoption

> 这些是基于代码的工程建议，不是作者承诺。可先复用思想、验证局部组件，再决定是否整体采用 runtime。

Async correction Expert placement Runtime optimization

Source map and remaining evidence gaps

Every technical slide links a reviewed snapshot and source location

- Read next: `entrypoints/llm.py`, `engine/llm_engine.py`, `csrc/engine/scheduler.hpp`, `worker/worker.py`.
- Model path: `v4_modules/attn.py`, `index_attn.py`, `mlp.py`, plus `worker/cache_engine.py`.
- Companions: harness search/file seams and Recipe token encoding. Full source index includes per-slide links.
- Limits: static review, no GPU benchmark, no audit of the unreleased cache/routing services, and no verified standalone search backend.

> 可以沿 source index 复查每个结论。若用户的 search 指另一个新仓库，需要具体名称后补充，不能用猜测填空。

Complete source index Reproducible authoring source Reviewed DeepInfer commit